

Middleware Pipeline پیشرفته در ASP.NET Core

Middlewares ← Roadmap: ASP.NET Core Basics

سطح: متوسط رو به پیشرفته

۳ مهر ۱۴۰۵

نسخه: 10.NET و 14 C#

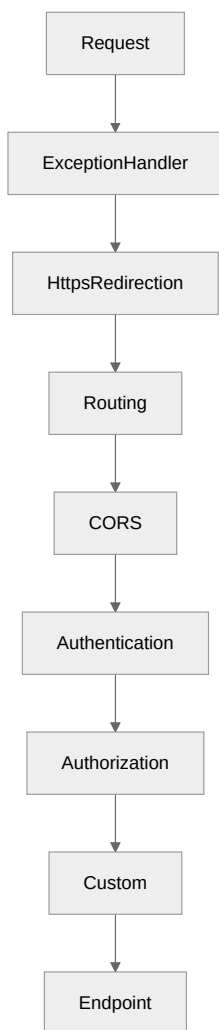
این اولین درس مسیر است. Filters و Authentication و Rate Limiting و Observability همه روی Pipeline سوار می‌شوند؛ پس اول باید آن را دقیق بشناسیم. کد این درس روی 10.0 NET SDK ساخته و با curl تست شده است.

۲ توضیح مفهومی

هر درخواست HTTP از یک زنجیره از `RequestDelegate` ها عبور می‌کند. هر Middleware سه انتخاب دارد:

- قبل از لایه بعدی کاری انجام دهد.
- `next` را صدا بزند تا درخواست جلو برود.
- بعد از برگشت پاسخ کاری انجام دهد.

اگر `next` صدا زده نشود، درخواست همان‌جا متوقف می‌شود. به این حالت Short-circuit می‌گویند.



پاسخ همین مسیر را برعکس برمی‌گردد. به همین دلیل ExceptionHandler اول قرار می‌گیرد تا خطای همه لایه‌های بعدی را بگیرد.

سه روش ساخت Middleware

1. Inline با `app.Use(...)` : برای منطق کوتاه و سریع.
2. Convention-based: کلاسی که در سازنده `RequestDelegate` می‌گیرد و متد `InvokeAsync` دارد. نمونه آن فقط یک‌بار در کل عمر برنامه ساخته می‌شود.
3. Factory-based: کلاسی که `IMiddleware` را پیاده می‌کند. در DI ثبت می‌شود و برای هر درخواست از همان Scope درخواست ساخته می‌شود.

۳ اهمیت و کاربرد

کارهای مشترک بین همه درخواست‌ها را یک‌بار و در یک نقطه پیاده می‌کنیم، نه در تک‌تک Controllerها. مثال‌ها: Correlation ID، لاگ زمان پاسخ، هدرهای امنیتی، مدیریت خطا و احراز هویت. این همان اصل DRY و Single Responsibility است.

Middleware یا Filter؟ اگر کار به همه درخواست‌ها مربوط است (حتی فایل استاتیک و 404) و به Model Binding نیاز ندارد، Middleware مناسب است. اگر به آرگومان‌های Action یا نتیجه آن نیاز دارید، از Filter یا Endpoint Filter استفاده کنید.

۴ مثال عملی

سناریو: یک API که به هر درخواست یک Correlation ID می‌دهد، زمان پاسخ را اندازه می‌گیرد و فقط مسیرهای `/api` را با API Key محافظت می‌کند.

ساخت پروژه

```
dotnet new web -n MiddlewareLab -f net10.0
cd MiddlewareLab
dotnet user-secrets init
dotnet user-secrets set "ApiKey" "dev-secret-change-me"
```

فایل `Middleware/RequestContext.cs` : سرویس Scoped برای اشتراک داده درخواست

```
namespace MiddlewareLab.Middleware;

// داده‌ای که در طول یک درخواست بین Middlewareها و Endpointها مشترک است
public interface IRequestContext
{
    string CorrelationId { get; set; }
}

public sealed class RequestContext : IRequestContext
{
}
```

```
public string CorrelationId { get; set; } = string.Empty;
}
```

فایل Middleware/CorrelationIdMiddleware.cs : روش Factory-based

```
using Microsoft.Extensions.Primitives;

namespace MiddlewareLab.Middleware;

// کلاس IMiddleware برای هر درخواست از همان Scope همان درخواست ساخته می‌شود؛
// پس تزریق سرویس Scoped در سازنده امن است.
public sealed class CorrelationIdMiddleware(
    IRequestContext requestContext,
    ILogger<CorrelationIdMiddleware> logger) : IMiddleware
{
    public const string HeaderName = "X-Correlation-Id";
    private const int MaxLength = 64;

    public async Task InvokeAsync(HttpContext context, RequestDelegate
next)
    {
        var correlationId =
ResolveId(context.Request.Headers[HeaderName]);

        requestContext.CorrelationId = correlationId;
        context.TraceIdentifier = correlationId;

        // هدرها فقط قبل از شروع ارسال پاسخ قابل تغییرند
        context.Response.OnStarting(() =>
        {
            context.Response.Headers[HeaderName] = correlationId;
            return Task.CompletedTask;
        });

        // همه لاگ‌های لایه‌های بعدی این Scope را خواهند داشت
        using (logger.BeginScope(new Dictionary<string, object> {
["CorrelationId"] = correlationId }))
        {
            await next(context);
        }
    }
}
```

```
// ورودی کاربر را اعتبارسنجی می‌کنیم تا Log Injection رخ ندهد
private static string ResolveId(StringValues incoming)
{
    var value = incoming.ToString();
    var isValid = value.Length is > 0 and <= MaxLength
        && value.All(c => char.IsAsciiLetterOrDigit(c) || c
== '-');

    return isValid ? value : Guid.NewGuid().ToString("N");
}
}
```

فایل `Middleware/RequestTimingMiddleware.cs` : روش Convention-based

```
using System.Diagnostics;

namespace MiddlewareLab.Middleware;

// یک بار ساخته می‌شود: فقط وابستگی‌های Singleton در سازنده
public sealed class RequestTimingMiddleware(
    RequestDelegate next,
    ILogger<RequestTimingMiddleware> logger)
{
    public const string HeaderName = "X-Elapsed-Ms";

    // وابستگی‌های Scoped به صورت پارامتر InvokeAsync تزریق می‌شوند
    public async Task InvokeAsync(HttpContext context, IRequestContext
requestContext)
    {
        var start = Stopwatch.GetTimestamp();

        context.Response.OnStarting(() =>
        {
            var elapsed =
Stopwatch.GetElapsedTime(start).TotalMilliseconds;
            context.Response.Headers[HeaderName] = elapsed.ToString("F1");
            return Task.CompletedTask;
        });

        try
        {
            await next(context);
        }
    }
}
```

```

    }
    finally
    {
        logger.LogInformation(
            "{Method} {Path} -> {StatusCode} in {ElapsedMs:F1} ms  
(cid={CorrelationId})",
            context.Request.Method,
            context.Request.Path,
            context.Response.StatusCode,
            Stopwatch.GetElapsedTime(start).TotalMilliseconds,
            requestContext.CorrelationId);
    }
}
}

```

فایل `Middleware/ApiKeyMiddleware.cs` : مقایسه زمان ثابت کلید

```

using System.Security.Cryptography;
using System.Text;

namespace MiddlewareLab.Middleware;

public sealed class ApiKeyMiddleware(RequestDelegate next, IConfiguration
configuration)
{
    public const string HeaderName = "X-API-Key";

    // کلید یک بار خوانده می شود؛ نبودن آن خطای شروع برنامه است (Fail fast)
    private readonly byte[] _expected = Encoding.UTF8.GetBytes(
        configuration["ApiKey"] ?? throw new
        InvalidOperationException("ApiKey is not configured."));

    public Task InvokeAsync(HttpContext context)
    {
        var provided =
        Encoding.UTF8.GetBytes(context.Request.Headers[HeaderName].ToString());

        // مدت FixedTimeEquals جلوی Timing Attack را می گیرد
        if (!CryptographicOperations.FixedTimeEquals(provided, _expected))
        {
            context.Response.StatusCode =
            StatusCodes.Status401Unauthorized;
        }
    }
}

```

```

        return Task.CompletedTask; // توقف زنجیره (Short-circuit) : next صدا زده نمی‌شود
    }

    return next(context);
}
}

```

فایل `Middleware/MiddlewareExtensions.cs` : یک نقطه برای ثبت و استفاده

```

namespace MiddlewareLab.Middleware;

public static class MiddlewareExtensions
{
    public static IServiceCollection AddRequestPipeline(this
    IServiceCollection services) =>
        services
            .AddScoped<IRequestContext, RequestContext>()
            .AddTransient<CorrelationIdMiddleware>(); // ثبت IMiddleware در DI
    // الزامی است

    public static IApplicationBuilder UseCorrelationId(this
    IApplicationBuilder app) =>
        app.UseMiddleware<CorrelationIdMiddleware>();

    public static IApplicationBuilder UseRequestTiming(this
    IApplicationBuilder app) =>
        app.UseMiddleware<RequestTimingMiddleware>();

    // شاخه شرطی که دوباره به Pipeline اصلی برمی‌گردد
    public static IApplicationBuilder UseApiKeyFor(this
    IApplicationBuilder app, PathString prefix) =>
        app.UseWhen(
            ctx => ctx.Request.Path.StartsWithSegments(prefix),
            branch => branch.UseMiddleware<ApiKeyMiddleware>());
}

```

فایل `Program.cs`

```

using MiddlewareLab.Middleware;

```

```

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddRequestPipeline();

var app = builder.Build();

// ترتیب مهم است: اول Correlation، سپس زمان سنجی، بعد امنیت
app.UseCorrelationId();
app.UseRequestTiming();
app.UseApiKeyFor("/api");

app.MapGet("/", (IRequestContext rc) => $"Hello! cid={rc.CorrelationId}");

app.MapGet("/api/orders", () => new[] { new { Id = 1, Total = 120_000m }
});

// توقف در سطح Routing با ShortCircuit (از .NET 8)
app.MapGet("/health", () => "ok").ShortCircuit();
app.MapShortCircuit(StatusCodes.Status404NotFound, "robots.txt",
"favicon.ico");

app.Run();

```

تست با curl (پورت را از خروجی `dotnet run` بردارید)

```

# 200 with X-Correlation-Id and X-Elapsed-Ms
curl -i http://localhost:5000/

# echoes the same id back: abc-123
curl -i -H "X-Correlation-Id: abc-123" http://localhost:5000/

# 401 without key, 200 with key
curl -i http://localhost:5000/api/orders
curl -i -H "X-API-Key: dev-secret-change-me"
http://localhost:5000/api/orders

# 200 but WITHOUT our headers (see section 5)
curl -i http://localhost:5000/health

```

۵ بررسی کد

چرا CorrelationId از IMiddleware استفاده می‌کند؟

چون `IRequestContext` یک سرویس Scoped است و می‌خواهیم در سازنده تزریق شود. `IMiddlewareFactory` این کلاس را برای هر درخواست از `context.RequestServices` می‌سازد.

دقت کنید: بدون `AddTransient<CorrelationIdMiddleware>()` در اولین درخواست خطا می‌گیرید.

چرا Timing سرویس Scoped را در InvokeAsync می‌گیرد؟

نمونه Convention-based فقط یک‌بار ساخته می‌شود. اگر سرویس Scoped را در سازنده آن تزریق کنید، در محیط Development خطای «Cannot resolve scoped service from root provider» می‌گیرید. در Production این خطا دیده نمی‌شود و یک Captive Dependency ساخته می‌شود.

متد Response.OnStarting

وقتی اولین بایت Body ارسال شد، هدرها قفل می‌شوند و تغییر آن‌ها `InvalidOperationException` می‌دهد. پس هدر را داخل Callback تنظیم می‌کنیم، نه بعد از `await next`.

یک نکته جالب: این Callback ها به ترتیب LIFO اجرا می‌شوند. برای همین در خروجی تست، هدر `X-Elapsed-Ms` قبل از `X-Correlation-Id` آمد.

بلوک try/finally

تضمین می‌کند حتی اگر لایه بعدی Exception بدهد، زمان پاسخ لاگ شود.

تفاوت MapWhen و UseWhen

شاخه UseWhen بعد از اجرا به Pipeline اصلی برمی‌گردد و به Endpoint می‌رسد. اما Map و MapWhen یک Pipeline جدا و بی‌بازگشت می‌سازند.

چرا health هدرهای ما را ندارد؟

اگر UseRouting را خودتان صدا نزنید، WebApplication آن را خودکار در ابتدای Pipeline قرار می‌دهد. متد ShortCircuit() درست بعد از Routing اجرا می‌شود. پس همه Middleware های ما و حتی Authentication رد می‌شوند.

نتیجه: ShortCircuit برای Health Check و robots.txt عالی است، ولی هرگز برای Endpoint محافظت‌شده از آن استفاده نکنید.

۶ نکات حرفه‌ای

اشتباهات رایج

- ترتیب غلط، مثلاً UseAuthorization قبل از UseAuthentication، یا CORS بعد از Authorization. این باگ امنیتی هیچ خطایی نشان نمی‌دهد.
- نگه‌داشتن داده مربوط به درخواست در فیلد کلاس Convention-based. چون نمونه بین همه درخواست‌ها مشترک است، Race Condition و نشت داده بین کاربرها رخ می‌دهد.
- نوشتن در Response بعد از await next بدون بررسی Response.HasStarted.
- اعتماد به هدر ورودی بدون اعتبارسنجی، که به Log Injection و Header Injection منجر می‌شود.
- مقایسه کلید با == که در برابر Timing Attack آسیب‌پذیر است. همان مفهومی که در یادداشت‌های Password Reset دیدید.

Performance

- Middleware روی هر درخواست اجرا می‌شود. از I/O همگام و تخصیص حافظه بی‌دلیل پرهیز کنید.

- در روش Inline، نسخه‌ای را انتخاب کنید که `next` از نوع `RequestDelegate` است: `app.Use(async (ctx, next) => await next(ctx))`. نسخه `Func<Task>` برای هر درخواست دو تخصیص حافظه اضافه دارد.

- `IMiddleware` برای هر درخواست یک شیء جدید می‌سازد. در مسیرهای بسیار پرتراфик، روش Convention-based با تزریق در `InvokeAsync` ارزان‌تر است.

Security

- `API Key` در این مثال فقط آموزشی است. در پروژه واقعی یک `AuthenticationHandler` بنویسید تا با `[Authorize]` و `Policy`ها یکپارچه شود.
- کلید را هرگز در `appsettings.json` نگذارید. در توسعه از `User Secrets` و در `Production` از `Key Vault` یا متغیر محیطی استفاده کنید.

۷ مقایسه روش‌ها

روش	طول عمر نمونه	تزریق Scoped	ثبت در DI	مناسب برای
Inline	یک‌بار	از <code>ctx.RequestServices</code>	ندارد	منطق چندخطی
Convention-based	یک‌بار	پارامتر <code>InvokeAsync</code>	ندارد	مسیرهای پرتراфик
<code>IMiddleware</code>	هر درخواست	سازنده	لازم	<code>Strong typing</code> و تست‌پذیری
<code>MVC Filter</code>	بسته به نوع ثبت	بله	بسته به نوع	منطق وابسته به <code>Controller</code> و <code>Action</code>
<code>Endpoint Filter</code>	برای هر <code>Endpoint</code>	بله	ندارد	<code>Minimal API</code>

۸ سوال مصاحبه

سوال ۱ (Senior): یک Middleware در سازنده خود ApplicationDbContext می‌گیرد. در

Production گاهی خطای «A second operation was started on this context» دیده می‌شود. مشکل چیست و چگونه حل می‌شود؟

Middleware از نوع Convention-based فقط یک‌بار ساخته می‌شود. پس DbContext که Scoped است داخل آن گیر می‌افتد (Captive Dependency) و بین همه درخواست‌های همزمان مشترک می‌شود. DbContext هم Thread-safe نیست. راه‌حل‌ها:

1. DbContext را به پارامترهای InvokeAsync منتقل کنید.
2. کلاس را به IMiddleware تبدیل و در DI ثبت کنید.
3. اگر کار خارج از چرخه درخواست است، از IDbContextFactory یا IServiceScopeFactory با Scope مستقل استفاده کنید.

سوال ۲: تفاوت Use و Run و Map و MapWhen و UseWhen چیست؟

- Use : لایه‌ای که می‌تواند next را صدا بزند.
- Run : انتهای زنجیره است و next ندارد.
- Map : بر اساس پیشوند مسیر یک Pipeline جدا می‌سازد و پیشوند را به PathBase منتقل می‌کند.
- MapWhen : مثل Map است ولی با یک شرط دلخواه.
- UseWhen : شاخه شرطی که به Pipeline اصلی برمی‌گردد، به شرطی که داخل شاخه Short-circuit نشود.

۹ تمرین روزانه

یک `SecurityHeadersMiddleware` بسازید که هدرهای `X-Content-Type-Options` و `X-Frame-Options` و `Content-Security-Policy` را اضافه کند.

□ مقادیر از `IOptions<SecurityHeadersOptions>` بیاید و با `ValidateOnStart` اعتبارسنجی شود (ادامه درس Options Pattern شما).

□ هدرها داخل `OnStarting` تنظیم شوند و برای مسیر `/swagger` با `UseWhen` غیرفعال شوند.

□ با `WebApplicationFactory<Program>` یک `Integration Test` بنویسید که وجود هدرها را بررسی کند.

□ سوال فکری: آیا `/health` هم این هدرها را می‌گیرد؟ چرا؟

۱۰ جمع‌بندی

- ترتیب ثبت همان ترتیب اجراست و پاسخ برعکس برمی‌گردد.
 - روش `Convention-based` یک‌بار ساخته می‌شود؛ سرویس `Scoped` را فقط در `InvokeAsync` بگیرید.
 - برای هر درخواست ساخته می‌شود و باید در `DI` ثبت شود.
 - هدر پاسخ را در `Response.OnStarting` تنظیم کنید.
 - `UseWhen` به `Pipeline` برمی‌گردد؛ `Map` و `MapWhen` برنمی‌گردند.
 - `ShortCircuit()` همه `Middleware`های بعد از `Routing` را رد می‌کند، از جمله `Authentication`.
- درس بعدی (پیشنهادی): `Dependency Injection` عمیق، شامل `Lifetime`ها، `Keyed Services` و `Captive Dependency`.

منبع رسمی: [Write custom ASP.NET Core middleware](#) · نقشه راه: [roadmap.sh/aspnet-core](#)